

Symbiosis Institute of Technology, Nagpur

DEPARTMENT OF COMPUTER SCIENCE • JAVASCRIPT PROGRAMMING LAB

Student: Tanmay Sankulwar

PRN: 24070521058

Course: Semester V (B.Tech)

EXPERIMENT 8

Student Name: Tanmay Sankulwar
PRN: 24070521058
File Path: EXP8/index.html , EXP8/Casestudy/

Experiment Title

Gym Admission Form with Interactive Field Validations & E-Commerce Product Filter

Objective

- To implement client-side interactive form validation using modern JavaScript event listeners (`input` , `blur` , `change` , and `submit`).
- To validate diverse user inputs (names, age ranges, email addresses, and phone numbers) using Regular Expressions (`RegExp`) and conditional boundary checks.
- To provide immediate visual feedback by rendering inline error notifications and preventing invalid form submission.
- To implement a dynamic e-commerce product filtering system utilizing array manipulation (`filter` , `forEach`), case-insensitive search matching, category selection, and dynamic DOM rendering.

Software / Tools Required

- Operating System:** Windows 10 / 11
- Development Environment:** Visual Studio Code / Antigravity IDE
- Web Browser:** Google Chrome / Modern Web Browser
- Languages & Technologies:** HTML5, CSS3, JavaScript (ES6+)

Program / Task

Task 8.a — Gym Admission Form with Real-Time Validation

File: EXP8/index.html

The practical task requires creating an interactive **Gym Admission Form** enclosed in a card-style container. The form validates all user inputs in real time: - **Full Name:** Validated on the `input` event using regex `/^[A-Za-z ]+$/` to ensure only alphabetic characters and spaces are entered. - **Age:** Validated on the `blur` event to ensure applicants fall within the eligible range of \$16 \le \text{Age} \le 60\$. - **Email Address:** Validated on the `input` event using regex `/^[^\\s@]+@[\\s@]+\\.\\[\\s@]+$/` . - **Mobile Number:** Validated on the `input` event using regex `/^\\d{10}$/` ensuring

an exact 10-digit numeric sequence. - **Membership Plan:** Validated on the `change` event to ensure a valid plan (Monthly, Quarterly, or Yearly) is selected from the dropdown menu. - **Form Submission:** Intercepts the `submit` event using `event.preventDefault()`, checks that all error indicators are cleared and a plan is chosen, and displays a success confirmation message "Gym Admission Successful!".

Source Code

Main Application — Gym Admission Form ([EXP8/index.html](#))

```
<!DOCTYPE html>
<html>

<head>
  <title>Gym Admission Form</title>

  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f2f2f2;
    }

    .container {
      width: 400px;
      margin: 40px auto;
      padding: 20px;
      background: white;
      border-radius: 10px;
      box-shadow: 0 0 10px gray;
    }

    h2 {
      text-align: center;
    }

    label {
      display: block;
      margin-top: 10px;
    }

    input,
    select {
      width: 100%;
      padding: 8px;
      margin-top: 5px;
    }

    .error {
      color: red;
      font-size: 13px;
    }

    .success {
      color: green;
      font-size: 14px;
      text-align: center;
    }

    button {
      width: 100%;
      padding: 10px;
      margin-top: 15px;
      background: green;
      color: white;
      border: none;
      cursor: pointer;
    }
  </style>
</head>

<body>
```

```
</style>
</head>

<body>

  <div class="container">

    <h2>Gym Admission Form</h2>

    <form id="gymForm">

      <label>Full Name</label>
      <input type="text" id="name">
      <span id="nameError" class="error"></span>

      <label>Age</label>
      <input type="number" id="age">
      <span id="ageError" class="error"></span>

      <label>Email</label>
      <input type="email" id="email">
      <span id="emailError" class="error"></span>

      <label>Mobile Number</label>
      <input type="text" id="mobile">
      <span id="mobileError" class="error"></span>

      <label>Membership Plan</label>

      <select id="plan">
        <option value="">Select Plan</option>
        <option>Monthly</option>
        <option>Quarterly</option>
        <option>Yearly</option>
      </select>

      <span id="planError" class="error"></span>

      <button type="submit">Submit</button>

      <p id="result" class="success"></p>

    </form>

  </div>

<script>

  document.getElementById("name").addEventListener("input", function () {

    let name = this.value;

    if (/^[A-Za-z ]+$/ .test(name)) {
      document.getElementById("nameError").innerHTML = "";
    }
    else {
      document.getElementById("nameError").innerHTML =
        "Only letters allowed.";
    }
  });

  document.getElementById("age").addEventListener("blur", function () {

    let age = this.value;

    if (age >= 16 && age <= 60) {
      document.getElementById("ageError").innerHTML = "";
    }
  })
}
```

```
        else {
            document.getElementById("ageError").innerHTML =
                "Age must be between 16 and 60.";
        }
    });

document.getElementById("email").addEventListener("input", function () {

    let email = this.value;

    let pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    if (pattern.test(email)) {
        document.getElementById("emailError").innerHTML = "";
    }
    else {
        document.getElementById("emailError").innerHTML =
            "Invalid email.";
    }
});

document.getElementById("mobile").addEventListener("input", function () {

    let mobile = this.value;

    if (/^\d{10}$/.test(mobile)) {
        document.getElementById("mobileError").innerHTML = "";
    }
    else {
        document.getElementById("mobileError").innerHTML =
            "Enter 10-digit mobile number.";
    }
});

document.getElementById("plan").addEventListener("change", function () {

    if (this.value == "") {
        document.getElementById("planError").innerHTML =
            "Please select a plan.";
    }
    else {
        document.getElementById("planError").innerHTML = "";
    }
});

document.getElementById("gymForm").addEventListener("submit", function (e) {

    e.preventDefault();

    if (
        document.getElementById("nameError").innerHTML == "" &&
        document.getElementById("ageError").innerHTML == "" &&
        document.getElementById("emailError").innerHTML == "" &&
        document.getElementById("mobileError").innerHTML == "" &&
        document.getElementById("plan").value != ""
    ) {

        document.getElementById("result").innerHTML =
            "Gym Admission Successful!";
    }
});
```

```
    }
    else {

        document.getElementById("result").innerHTML = "";

        alert("Please correct the errors before submitting.");

    }

});

</script>

</body>

</html>
```

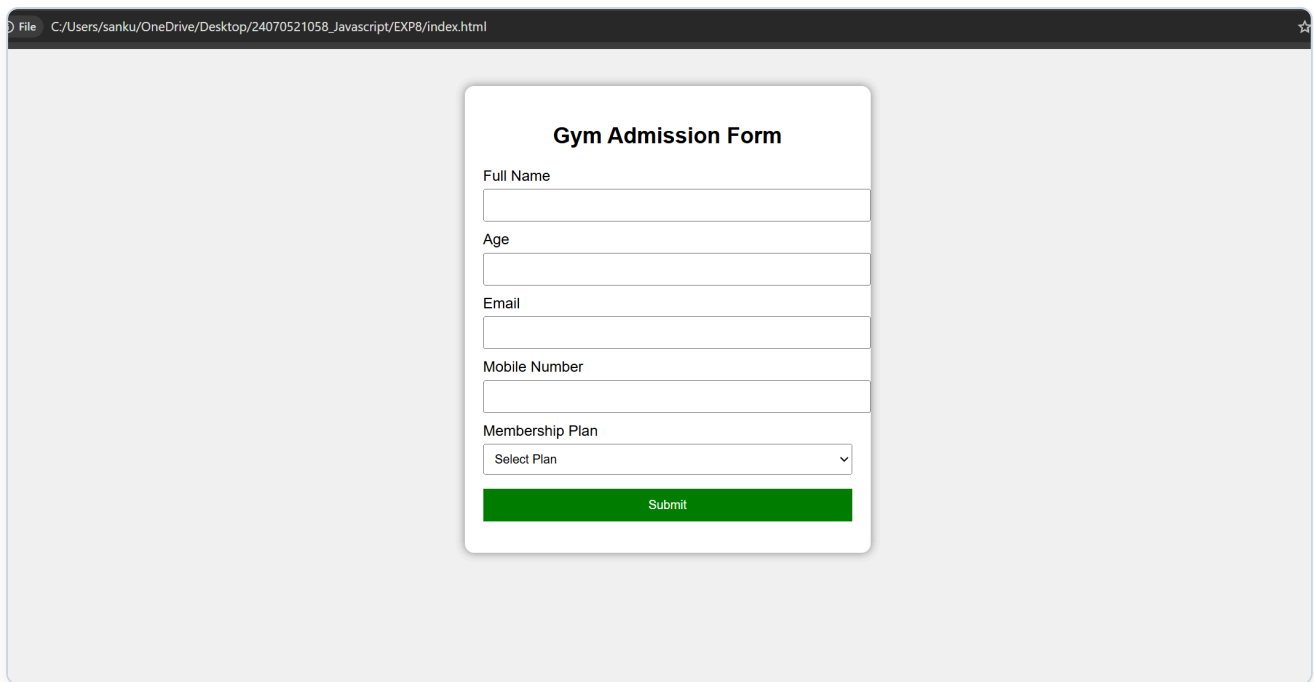
Explanation / Concepts

- Real-Time Input Monitoring (`input` event):** - The `input` event fires immediately upon each keystroke or character change, allowing instant validation feedback without requiring the user to unfocus the element. - For example, `/^[A-Za-z ]+$/ .test(name)` validates that the entered name contains exclusively uppercase letters, lowercase letters, or spaces. - `/\d{10}$/ .test(mobile)` validates that the mobile number contains exactly 10 numeric digits.
- Boundary Validation on Loss of Focus (`blur` event):** - The `blur` event triggers when the user navigates away from the input field. - The age check evaluates numerical boundaries (`$16 \le \text{Age} \le 60`) once the user has finished typing their age.
- Dropdown Selection Validation (`change` event):** - The `change` event verifies that the dropdown option chosen has a non-empty string value (`this.value != ""`).
- Event Interception & Submission Control (`submit` event):** - The `e.preventDefault()` method cancels default HTTP form submission, preventing page reloads. - Form submission conditions verify that all error spans are empty and a valid plan is active before displaying the positive confirmation message.

Output

- Initial Rendering:** A neat, centered white card displaying input fields for Full Name, Age, Email, Mobile Number, a Membership Plan dropdown, and a green Submit button.
- Interactive Validation:** - If invalid characters are typed in the Name field, `"Only letters allowed."` appears in red. - If an out-of-range age is provided, `"Age must be between 16 and 60."` appears on blur. - If an improper email format is typed, `"Invalid email."` appears. - If the mobile number does not match 10 digits, `"Enter 10-digit mobile number."` appears.
- Successful Submission:** When all validation criteria are met, clicking **Submit** displays `"Gym Admission Successful!"` in green beneath the submit button.

Screenshot

A screenshot of a web browser window displaying a 'Gym Admission Form'. The browser's address bar shows the file path 'C:/Users/sanku/OneDrive/Desktop/24070521058_Javascript/EXP8/index.html'. The form is centered on a light gray background and contains the following fields: 'Full Name' (text input), 'Age' (text input), 'Email' (text input), 'Mobile Number' (text input), 'Membership Plan' (dropdown menu with 'Select Plan' as the current selection), and a green 'Submit' button at the bottom.

Gym Admission Form Output

Case Study

Case Study — E-Commerce Product Filter with Live Search & Categories

Files: EXP8/Casestudy/index.html , EXP8/Casestudy/script.js , EXP8/Casestudy/style.css

The case study expands upon event-driven architecture and input validation by implementing an interactive **E-Commerce Product Filter** that allows users to filter products dynamically by keyword search and product category.

Key Features & Architecture:

- **Product Dataset:** An array of product objects containing `name` , `category` , and `price` .
- **Search Sanitization:** Validates live keystrokes against `/^[A-Za-z ]*$/` to ensure only clean search terms are accepted; if special characters or numbers are entered, an error prompt `"Only letters and spaces are allowed."` is rendered.
- **Combined Array Filtering (`filter()`):** Filters products by case-insensitive name matching (`name.toLowerCase().includes(searchText)`) and category selection (`selectedCategory === "All" || product.category === selectedCategory`).
- **Dynamic DOM Rendering (`forEach()`):** Creates structured product card elements with `.product` styling, displays formatted pricing (`₹price`), and displays a friendly `"No products found"` message if the filtered result is empty.
- **Reset Functionality:** The **Clear Filter** button resets search inputs, restores category to `"All"` , clears errors, and re-renders the complete product collection.

```
// Excerpt from EXP8/Casestudy/script.js
function filterProducts() {
  let searchText = searchBox.value.toLowerCase();
  let selectedCategory = category.value;

  if (!/^[A-Za-z ]*$/ .test(searchBox.value)) {
    errorMessage.innerHTML = "Only letters and spaces are allowed.";
    productList.innerHTML = "";
    noProducts.innerHTML = "";
  }
}
```

```
        return;
    }

    errorMessage.innerHTML = "";

    let filteredProducts = products.filter(function (product) {
        let matchesSearch = product.name.toLowerCase().includes(searchText);
        let matchesCategory = selectedCategory === "All" || product.category === selectedCategory;
        return matchesSearch && matchesCategory;
    });

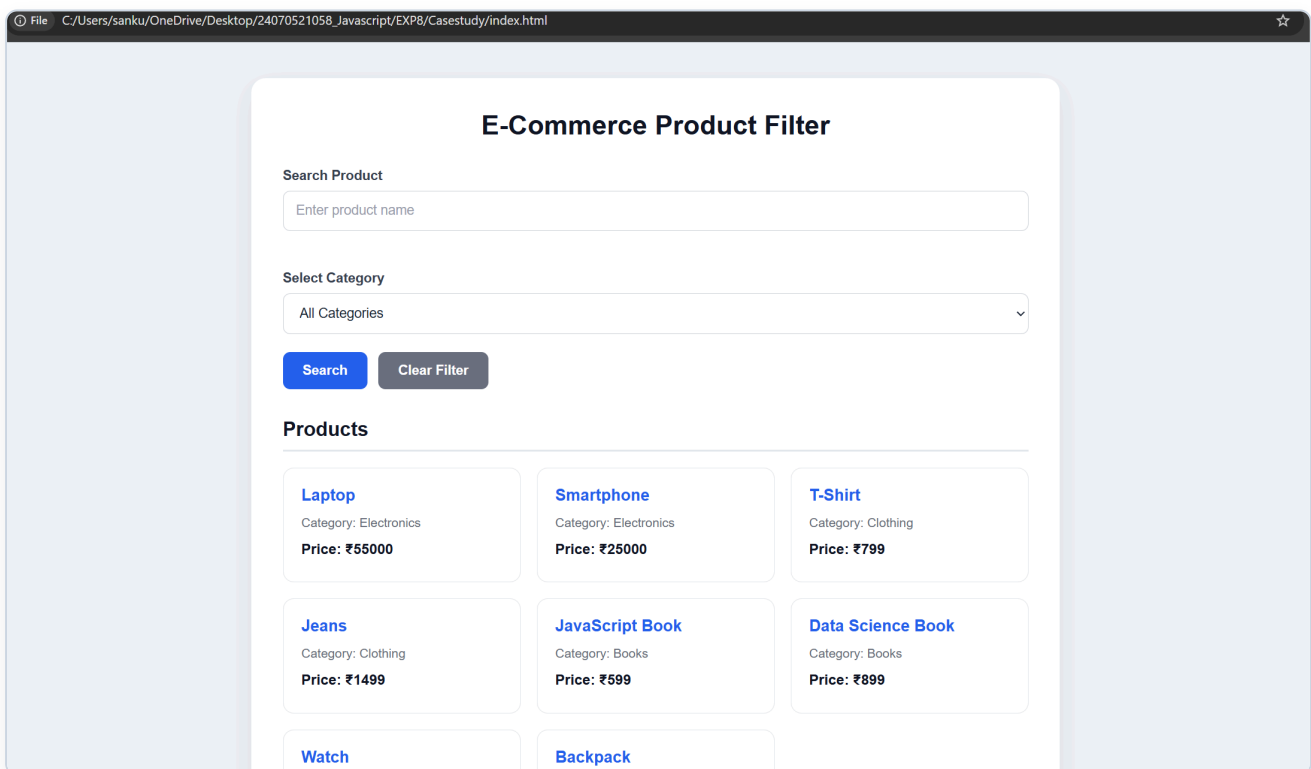
    displayProducts(filteredProducts);
}

searchBox.addEventListener("input", function () {
    filterProducts();
});

category.addEventListener("change", function () {
    filterProducts();
});

clearBtn.addEventListener("click", function () {
    searchBox.value = "";
    category.value = "All";
    errorMessage.innerHTML = "";
    displayProducts(products);
});
```

Case Study Screenshot



E-Commerce Product Filter Output

Result / Conclusion

Experiment 8 was successfully implemented and verified. The practical demonstrated comprehensive client-side form validation using JavaScript event listeners (`input` , `blur` , `change` , `submit`) alongside Regular Expression pattern

matching. The case study demonstrated dynamic catalog searching and filtering using functional array methods (`filter` , `forEach`), DOM node creation, and responsive CSS Grid layout design.